



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

Trabajo fin de Grado

GIC, GIS y GITI

Especificaciones de formato

**Realizado por
(ponente): José Ra. Portillo Fernández**

**Dirigido por
Junta de Centro**

**Departamento
Comisión de Trabajos fin de Grado de la E.T.S.I.I.**

Sevilla, 1 de marzo de 2017 (v.0.91)

Índice general

I Introducción y Conceptos

1	Introducción	2
1.1	Contexto	2
1.2	Motivación	2
1.3	Objetivos	3
1.3.1	Objetivo general	3
1.3.2	Objetivos técnicos	3
1.3.3	Objetivos académicos y personales	3
1.4	Estructura de la memoria	4
2	Preparación del proyecto	5
2.1	Estudio de las tecnologías y toma de decisiones	5
2.1.1	Decisión de frameworks conversacionales	5
2.1.2	Decisión de base de datos	7
2.1.3	Decisión de API de LLM	9
2.2	Infraestructura del sistema	12
2.2.1	Arquitectura del sistema	12
2.2.2	Componentes tecnológicos del sistema	13
2.3	Casos de uso principales	13
2.3.1	Consulta sobre horarios de clases	13
2.3.2	Consulta sobre documentación necesaria para trámites	13
2.3.3	Consulta sobre datos de contacto y tutorías del profesorado	14
2.3.4	Consulta sobre información de una asignatura	15
2.3.5	Consulta sobre normas y documentación de la Universidad	16
3	Conceptos iniciales	17
3.1	Sistema conversacional y componentes de Rasa	17
3.2	Procesamiento del lenguaje natural	17
3.3	Gestión del diálogo	17
3.4	Backend y persistencia de datos	17

II Metodología y Planificación

4	Metodología	19
4.1	SCRUM ajustado en la ejecución	19
4.2	Herramientas	19
5	Planificación	20
5.1	Backlog	20
5.2	Planificación de sprints	20
5.3	Cronograma	20
	Referencias	21

Índice de figuras

Índice de tablas

2.1	Comparación de frameworks conversacionales	8
2.2	Comparación de bases de datos	9
2.3	Comparación de APIs de LLM	12
2.4	Componentes tecnológicos del sistema	13

Índice de código

Parte I

Introducción y Conceptos

Introducción

Este proyecto tiene como finalidad el diseño y desarrollo de un chatbot inteligente orientado a facilitar el acceso a información académica y administrativa en la Universidad de Sevilla, mediante técnicas de procesamiento del lenguaje natural y recuperación aumentada por generación (RAG).

1.1– Contexto

En la actualidad, el alumnado de la Universidad de Sevilla se enfrenta a una notable dispersión informativa. La información académica y administrativa se encuentra repartida entre múltiples plataformas y páginas web, cada una gestionada de forma independiente por distintas facultades, servicios y departamentos. Esta fragmentación dificulta el acceso eficiente a datos esenciales como horarios, contacto con profesorado, requisitos para trámites o metodologías de asignaturas.

Además, muchos contenidos relevantes no se presentan de forma unificada ni están estructurados para su consulta directa, sino que requieren navegar por documentos PDF (como proyectos docentes), correos informativos, páginas institucionales específicas u otros portales como Sevius. Esta situación resulta especialmente compleja para estudiantes de nuevo ingreso o internacionales, que no siempre conocen el funcionamiento interno del entorno universitario y, en muchos casos, deben enfrentarse también a la barrera del idioma, lo que dificulta aún más la comprensión de la información disponible.

La ausencia de un punto de acceso único, interactivo y orientado al usuario final pone de manifiesto la necesidad de una herramienta conversacional que permita acceder a la información de manera contextual, simplificada y guiada. En este sentido, un chatbot capaz de interpretar consultas abiertas en lenguaje natural y conectarse con diferentes fuentes de información representa una solución innovadora, alineada con los principios de accesibilidad, automatización y modernización de los servicios universitarios.

1.2– Motivación

La motivación principal de este proyecto surge de la necesidad de centralizar el acceso a información académica dispersa y mejorar la experiencia del estudiante en su interacción con los sistemas universitarios. Los estudiantes, especialmente aquellos de nuevo ingreso o de programas de intercambio, se enfrentan con frecuencia a dificultades para localizar información básica como horarios de clase, datos de contacto del profesorado o documentación necesaria para realizar trámites administrativos.

Esta problemática no solo genera frustración y pérdida de tiempo, sino que también puede afectar negativamente al rendimiento académico y a la integración del estudiante en la comunidad

universitaria. La dispersión de la información en múltiples fuentes web, documentos PDF y portales específicos dificulta la autonomía informativa del alumnado, que debe dedicar un esfuerzo significativo a buscar respuestas que deberían estar disponibles de forma inmediata.

Por tanto, el desarrollo de un asistente conversacional capaz de responder de forma rápida, precisa y en lenguaje natural constituye una mejora sustancial en la calidad del servicio universitario, promoviendo la autonomía del estudiante y reduciendo la carga administrativa de los servicios de atención.

1.3– Objetivos

1.3.1. Objetivo general

El objetivo principal de este proyecto es diseñar y desarrollar un chatbot inteligente que proporcione a los estudiantes de la Universidad de Sevilla una vía centralizada, accesible y eficiente para consultar información académica y administrativa relevante. La solución pretende integrar capacidades de comprensión del lenguaje natural mediante modelos de Inteligencia Artificial Generativa, junto con búsquedas en bases de datos estructuradas y vectorizadas, permitiendo resolver dudas frecuentes relacionadas con horarios, profesorado, documentación, normativas o localización de espacios universitarios. El asistente está especialmente orientado a facilitar la autonomía informativa de estudiantes de nuevo ingreso, internacionales o aquellos menos familiarizados con los procedimientos institucionales de la Universidad de Sevilla.

1.3.2. Objetivos técnicos

Los objetivos técnicos específicos del proyecto incluyen:

- Implementar un sistema conversacional basado en Rasa que permita gestionar flujos de diálogo complejos y contextuales.
- Integrar la API de Gemini para la detección de intenciones y generación de respuestas enriquecidas mediante lenguaje natural.
- Diseñar y desarrollar una base de datos híbrida en Supabase que combine información estructurada (horarios, profesorado, trámites) con embeddings vectoriales para búsquedas semánticas.
- Desarrollar acciones personalizadas en Python que permitan consultar datos estructurados y realizar búsquedas por similitud en documentación académica.
- Implementar un frontend web en React que ofrezca una interfaz intuitiva y accesible para la interacción con el chatbot.
- Garantizar el cumplimiento de normativas de privacidad (RGPD) mediante despliegue local y control de acceso a datos sensibles.

1.3.3. Objetivos académicos y personales

Desde el punto de vista académico y personal, este proyecto permite:

- Aplicar conocimientos adquiridos durante el Grado en Ingeniería del Software en un proyecto real de desarrollo de software.
- Adquirir experiencia práctica en el diseño de sistemas conversacionales y en la integración de modelos de inteligencia artificial generativa.

- Profundizar en el uso de técnicas de Retrieval Augmented Generation (RAG) y bases de datos vectoriales.
- Desarrollar competencias en metodologías ágiles aplicadas a proyectos individuales.
- Contribuir a la mejora de los servicios universitarios mediante una solución tecnológica innovadora y accesible.
- Demostrar la capacidad de diseñar, implementar y documentar un sistema completo de forma autónoma.

1.4– Estructura de la memoria

Esta memoria se organiza en dos partes principales:

Parte I: Introducción y Conceptos, donde se contextualiza el proyecto, se justifica su necesidad, se definen los objetivos y se describen los conceptos técnicos fundamentales relacionados con sistemas conversacionales, procesamiento del lenguaje natural y arquitectura del sistema.

Parte II: Metodología y Planificación, que detalla la metodología de desarrollo empleada (SCRUM adaptado), las herramientas utilizadas y la planificación temporal del proyecto, incluyendo el backlog de tareas y la distribución en sprints.

Preparación del proyecto

En esta fase previa se llevó a cabo un análisis exhaustivo de las tecnologías disponibles para construir el sistema conversacional, evaluando alternativas en función de criterios como coste, facilidad de integración, capacidad de despliegue local y compatibilidad con técnicas de RAG. Asimismo, se definió la arquitectura técnica del sistema y se identificaron los casos de uso principales que guiarían el desarrollo del proyecto.

2.1– Estudio de las tecnologías y toma de decisiones

2.1.1. Decisión de frameworks conversacionales

Frameworks analizados

Se han considerado distintos frameworks de chatbot **open source** que cumplen con los requisitos de despliegue local, integración en Python y soporte para RAG. Los principales son:

- **Rasa**, centrado en Python y con gran control de flujo.
- **Botpress (OSS)**, que destaca por su editor visual de flujos.
- **DeepPavlov**, más orientado a NLP avanzado con módulos reutilizables.
- **OpenDialog**, centrado en diseño visual y contextos conversacionales.
- **Tock** y **Botonic**, opciones alternativas con características interesantes, aunque con menor adopción o diferente enfoque tecnológico.

Comparación detallada

Flujo conversacional El flujo conversacional se refiere a la capacidad del framework para **definir, controlar y gestionar el diálogo entre el usuario y el sistema**. Incluye cómo se modelan las intenciones, las respuestas, los contextos y la memoria de la conversación, así como el grado de flexibilidad para manejar **diálogos no lineales, interrupciones o escenarios complejos**. Un buen manejo del flujo conversacional permite que el chatbot no solo siga guiones rígidos, sino que pueda adaptarse dinámicamente a las necesidades del usuario, manteniendo coherencia y contexto a lo largo de la interacción.

- **Rasa** ofrece máximo control mediante historias y reglas, permitiendo modelar diálogos complejos y contextuales.

- **Botpress** facilita el diseño con nodos visuales, aunque está más orientado a flujos rígidos y predefinidos.
- **DeepPavlov** permite construir agentes multi-skill, pero requiere definir manualmente los pipelines.
- **OpenDialog** sobresale en el modelado de escenarios y contextos, aunque su dependencia de PHP limita la integración con Python.
- **Tock/Botonic** permiten definir historias o rutas conversacionales, pero con menor madurez que Rasa.

Testing La capacidad de testing en un framework de chatbots hace referencia a las **herramientas disponibles para validar la calidad y el correcto funcionamiento del agente conversacional**. Esto incluye desde pruebas unitarias (validación de intenciones, entidades y respuestas individuales) hasta pruebas end-to-end (que simulan conversaciones completas para comprobar flujos y contextos). Un buen soporte de testing permite **automatizar la detección de errores, reducir el riesgo de alucinaciones o respuestas incoherentes y asegurar la estabilidad** del sistema a medida que evoluciona. Cuanto más integrado esté el testing en el framework, más fácil será mantener y escalar el chatbot con garantías de calidad.

- **Rasa** incorpora herramientas para pruebas unitarias y end-to-end, automatizando la validación de intenciones y flujos.
- **Botpress** ofrece principalmente un simulador manual, sin suite de pruebas automatizadas robusta.
- **DeepPavlov** no incluye framework específico, por lo que las pruebas deben hacerse con scripts en Python.
- **OpenDialog** y **Botonic** dependen casi exclusivamente de pruebas manuales.

Compatibilidad con RAG y embeddings La compatibilidad con **RAG (Retrieval Augmented Generation)** y el uso de **embeddings** mide hasta qué punto un framework permite integrar modelos de lenguaje con bases de datos vectoriales para mejorar las respuestas. Esta característica es clave cuando se trabaja con **información documental extensa o cambiante**. Un buen soporte en este ámbito asegura que el chatbot pueda **ofrecer respuestas actualizadas, precisas y fundamentadas en datos externos**, reduciendo el riesgo de respuestas inventadas (alucinaciones).

- **Rasa** se integra de forma natural con bases vectoriales (pgvector, Qdrant, FAISS), permitiendo retrieval augmented generation.
- **Botpress** dispone de un módulo de *Knowledge Base* que admite cargar documentos, aunque menos flexible en personalización.
- **DeepPavlov** destaca en Q&A y FAQ matching, aunque el RAG debe construirse manualmente.
- **OpenDialog** y **Botonic** no tienen soporte nativo, siendo necesaria integración externa.

Despliegue local y privacidad El despliegue local y la gestión de la privacidad se refieren a la **posibilidad de ejecutar el framework en entornos controlados (on-premise o en servidores propios)** sin depender necesariamente de servicios en la nube. Esta característica es importante cuando se manejan **datos sensibles o confidenciales**, ya que permite aplicar políticas de seguridad personalizadas y cumplir con normativas como el RGPD. Además, la facilidad de despliegue (con Docker, instaladores nativos o stacks ligeros) influye directamente en la **rapidez de la puesta en marcha y en la capacidad de mantener el control total sobre la información del usuario**.

- **Rasa** y **Botpress** son fácilmente desplegables en local mediante Docker o instalación directa.
- **DeepPavlov** funciona como librería Python o servicio REST.
- **OpenDialog** requiere un stack PHP más pesado.
- **Tock** es estable en entornos locales, mientras que Botonic se orienta a proyectos en Node/React.

Comunidad y documentación La comunidad y la documentación de un framework son indicadores clave de su madurez, accesibilidad y sostenibilidad a largo plazo. Una comunidad activa ofrece foros, repositorios y soporte colaborativo que facilitan la resolución de problemas y el intercambio de buenas prácticas. Al mismo tiempo, una documentación clara, actualizada y completa reduce la curva de aprendizaje y permite implementar soluciones más rápido.

- **Rasa** tiene la comunidad más amplia y documentación extensa.
- **Botpress** dispone de foros y Discord activos, pero con menor alcance.
- **DeepPavlov** cuenta con una comunidad académica sólida, aunque más técnica.
- **OpenDialog** y **Botonic** tienen comunidades pequeñas y más limitadas.

Decisión final

El framework seleccionado es **Rasa**, ya que combina:

- Máximo control sobre el flujo conversacional.
- Capacidades de testing integradas y automatizables.
- Soporte probado para RAG y bases vectoriales.
- Despliegue local sencillo y seguro, compatible con RGPD.
- Amplia comunidad y documentación, lo que garantiza sostenibilidad del proyecto.

Tabla de comparación detallada de frameworks

2.1.2. Decisión de base de datos

Bases de Datos analizadas

Se revisaron diferentes opciones open source para almacenar tanto información estructurada como embeddings:

- **Supabase (Postgres + pgvector).**
- **PostgreSQL autogestionado con pgvector.**
- **Appwrite, PocketBase, Directus, Hasura.**
- Motores vectoriales dedicados: **Qdrant, Weaviate, Milvus, FAISS.**

Característica / Framework	Rasa	Botpress	DeepPavlov	OpenDialog	Tock	Botonic
Flujo conversacional	***	**	**	**	*	*
Testing	***	*	*	*	*	*
Compatibilidad con RAG y embeddings	***	**	**	*	*	*
Despliegue local y privacidad	***	***	**	*	**	**
Comunidad y documentación	***	**	**	*	*	*
Resultado global	* 15	* 10	* 9	* 6	* 6	* 6

Cuadro 2.1: Comparación de frameworks conversacionales

Comparación detallada

Modelo de datos El modelo de datos define cómo se organiza, almacena y consulta la información dentro de una base de datos o motor de persistencia. Dependiendo de la tecnología, este modelo puede ser relacional (tablas y relaciones SQL), orientado a documentos (estructuras tipo JSON), o especializado en vectores (espacios multidimensionales para embeddings). La elección del modelo es clave para garantizar la eficiencia, flexibilidad y escalabilidad del sistema, ya que influye directamente en cómo se estructuran los datos de los usuarios, los recursos académicos y los embeddings utilizados en búsquedas semánticas o RAG.

Búsqueda vectorial

- **Supabase/Postgres** soportan pgvector, adecuado para corpus medianos.
- **Qdrant, Weaviate y Milvus** son más eficientes en grandes volúmenes de vectores.
- **FAISS** funciona muy bien embebido en Python, pero no es una base de datos como tal.
- El resto (Appwrite, PocketBase, Directus, Hasura) carecen de soporte nativo y requieren integraciones manuales.

Despliegue local y facilidad El despliegue local y la facilidad de instalación miden la **complejidad técnica y los recursos necesarios para poner en marcha una base de datos en un entorno controlado**. Algunas soluciones ofrecen entornos ligeros y rápidos de ejecutar (como binarios únicos o imágenes Docker sencillas), mientras que otras requieren stacks más pesados y múltiples dependencias. Esta característica es importante porque impacta directamente en la **rapidez de la configuración inicial, la curva de aprendizaje y los costes de mantenimiento**.

- **Supabase y Postgres** se despliegan fácilmente con Docker.
- **PocketBase** es el más liviano (un solo binario).
- **Appwrite y Directus** requieren más dependencias.
- **Qdrant/Weaviate/Milvus** añaden complejidad, aunque escalables.

Seguridad y RGPD La seguridad y el cumplimiento normativo son aspectos fundamentales en la gestión de datos, especialmente cuando se trabaja con información sensible de estudiantes o usuarios. Esta característica evalúa los **mecanismos nativos de autenticación, autorización y control de acceso** que ofrece cada base de datos, así como la posibilidad de aplicar políticas de privacidad. Tecnologías que incluyen funciones como **Row-Level Security, roles definidos o autenticación integrada** facilitan el cumplimiento legal y reducen riesgos. En cambio, los motores que carecen de seguridad propia obligan a implementar medidas adicionales en la capa de aplicación, aumentando la complejidad y la responsabilidad del equipo de desarrollo.

- **Supabase** ofrece Row-Level Security (RLS) y autenticación JWT.
- **Postgres** tradicional depende de roles SQL.
- **Appwrite/Directus/Hasura** ofrecen mecanismos de auth integrados.
- **Motores vectoriales** carecen de autenticación propia, requiriendo protección adicional en la aplicación.

Comunidad y documentación La comunidad y la documentación determinan el **nivel de soporte, recursos y acompañamiento disponibles para desarrolladores**. Una base de datos con una comunidad amplia y activa facilita la resolución de problemas, la existencia de librerías y la mejora continua del sistema. Al mismo tiempo, una documentación clara, actualizada y con ejemplos prácticos acelera la adopción de la tecnología y reduce la curva de aprendizaje.

- **Postgres** es el más maduro y con mayor soporte.
- **Supabase** ha crecido mucho, con amplia documentación y ejemplos de IA.
- **Qdrant y Weaviate** cuentan con comunidades activas en el ámbito de vectores.
- **Appwrite, PocketBase, Directus** tienen comunidades menores pero en expansión.

Decisión final

La base de datos seleccionada es **Supabase (Postgres + pgvector)**, porque:

- Permite unificar datos estructurados y vectores en una sola plataforma.
- Ofrece autenticación, reglas de seguridad a nivel de fila y APIs listas para consumir desde Python y React.
- Facilita el despliegue local sin costes de licencias.
- Su comunidad y documentación reducen la curva de aprendizaje.
- Cumple los requisitos de RAG con un volumen de datos manejable para el caso académico.

Tabla de comparación de bases de datos

Característica / BD	Supabase	Postgres	Appwrite	PocketBase	Directus	Hasura	Qdrant	Weaviate	Milvus	FAISS
Modelo de datos	***	***	**	**	**	**	**	**	**	**
Búsqueda vectorial	**	**	*	*	*	*	***	***	***	**
Despliegue local y facilidad	***	***	**	***	**	**	**	**	**	**
Seguridad y RGPD	***	**	**	*	**	**	*	*	*	*
Comunidad y documentación	***	***	**	**	**	**	**	**	**	**
Resultado global	★ 14	★ 13	★ 9	★ 9	★ 9	★ 9	★ 10	★ 10	★ 10	★ 9

Cuadro 2.2: Comparación de bases de datos

2.1.3. Decisión de API de LLM

APIs analizadas

Se han considerado diferentes opciones de APIs de modelos de lenguaje de gran tamaño (LLMs) open source o con planes gratuitos/educativos:

- **Gemini (Google AI / DeepMind)**

- **OpenAI GPT (GPT-4o, GPT-4o mini)**
- **DeepSeek**
- **Hugging Face Inference API**
- **Cohere y Anthropic (Claude)**

Comparación detallada

Coste y accesibilidad El coste y la accesibilidad hacen referencia a la **disponibilidad económica y facilidad de uso de los modelos de lenguaje (LLMs)**. En un contexto académico, donde no suele existir presupuesto, es esencial contar con opciones que permitan **experimentar gratuitamente o con bajos costes iniciales**. Además del precio, influyen factores como los límites de uso en los planes gratuitos, la velocidad de respuesta, la estabilidad del servicio y la facilidad de registro o integración. Esta característica resulta clave para asegurar que el proyecto pueda desarrollarse de forma **sostenible y realista**, sin que los costes de las llamadas a la API se conviertan en una barrera.

- Gemini ofrece un plan gratuito con un número considerable de llamadas mensuales, lo que permite experimentar y probar sin coste, factor clave en un TFG sin presupuesto.
- OpenAI GPT tiene un coste asociado desde la primera llamada más allá de GPT-3.5, lo que lo hace menos viable económicamente.
- DeepSeek tiene una API gratuita, pero con menos garantías de estabilidad y soporte.
- Hugging Face API ofrece acceso a modelos open source, pero el plan gratuito tiene fuertes limitaciones de velocidad y peticiones.
- Claude (Anthropic) y Cohere requieren suscripción desde el inicio, menos atractivas en un contexto universitario.

Calidad y rendimiento La calidad y el rendimiento de un modelo de lenguaje se refieren a su capacidad para **comprender instrucciones, generar texto coherente y relevante, y mantener el contexto a lo largo de la conversación**. Estos aspectos suelen evaluarse mediante **pruebas comparativas de comprensión, razonamiento y generación en varios idiomas**. Además, influyen factores prácticos como la **velocidad de respuesta, la estabilidad del modelo y su habilidad para manejar entradas extensas sin pérdida de coherencia**. Una mayor calidad en este ámbito se traduce en **respuestas más útiles, precisas y naturales**, lo que resulta fundamental para un chatbot académico que debe manejar información variada con fiabilidad.

Estudios comparativos muestran que Gemini compite con GPT-4 en comprensión y generación de texto, especialmente en multilingüe y razonamiento.

- GPT sigue siendo referencia en benchmarks de calidad, pero con coste.
- DeepSeek ha demostrado buen rendimiento en ciertas tareas de razonamiento matemático, aunque en comprensión semántica y contexto extenso suele quedar por debajo de Gemini y GPT.
- Hugging Face API depende mucho del modelo seleccionado (ej. LLaMA, Mistral, Falcon), pero requieren más trabajo de fine-tuning y optimización para igualar a Gemini/GPT en calidad.

Integración y soporte técnico La integración y el soporte técnico se refieren a la **facilidad con la que un LLM puede incorporarse en el stack tecnológico existente** y al nivel de recursos disponibles para resolver problemas durante el desarrollo. Factores como la existencia de **SDKs oficiales, librerías en distintos lenguajes, ejemplos prácticos y documentación clara** determinan la rapidez de adopción. Además, contar con un ecosistema sólido y acuerdos institucionales (como convenios académicos) puede simplificar la integración y asegurar **mayor estabilidad a largo plazo**. Esta característica es clave, ya que permite centrar el esfuerzo en el desarrollo del chatbot sin tener que invertir demasiado tiempo en resolver barreras técnicas de conexión con el modelo.

- Gemini ofrece SDKs y buena integración en Python/JavaScript, lo que encaja perfectamente con tu stack (Python + React).
- GPT también tiene SDKs muy maduros, aunque con la barrera de costes.
- DeepSeek todavía no cuenta con ecosistema sólido.
- Hugging Face API es flexible pero requiere seleccionar y mantener modelos específicos.
- Gemini además está alineado con acuerdos académicos, como el que mantiene la **Universidad de Sevilla con Google** en diferentes ámbitos, lo que puede facilitar futuras integraciones oficiales del chatbot.

Privacidad y cumplimiento (RGPD) La privacidad y el cumplimiento normativo son esenciales al utilizar modelos de lenguaje que procesan datos de usuarios. Esta característica evalúa el **nivel de control sobre dónde y cómo se almacenan los datos**, si el servicio cumple con estándares legales de protección de información y si existe la opción de **despliegue on-premise**. Los LLMs en la nube suelen garantizar cumplimiento básico de GDPR, pero limitan el control directo. En cambio, los modelos auto-hospedados ofrecen **mayor soberanía sobre los datos**, aunque requieren más infraestructura y experiencia técnica. En un contexto universitario, suele ser más relevante encontrar un equilibrio entre **facilidad de uso, coste cero o reducido y garantías mínimas de privacidad**.

- Gemini y GPT ofrecen despliegues en la nube con cumplimiento GDPR, aunque el control on-premise es limitado.
- Los modelos de Hugging Face pueden auto-hospedarse, lo que da mayor control, pero a costa de infraestructura.
- En un contexto académico, la facilidad de uso y el plan gratuito de Gemini pesan más que el control absoluto.

Decisión final

La API seleccionada es **Gemini**, porque:

- Dispone de un plan gratuito generoso en número de llamadas.
- Ofrece calidad comparable a GPT-4 en muchos benchmarks, especialmente en español y razonamiento.
- Tiene SDKs oficiales para Python y JavaScript, encajando con el stack técnico del proyecto.
- Permite asegurar continuidad futura gracias a posibles acuerdos institucionales con la Universidad de Sevilla.
- Representa la opción más equilibrada entre calidad, coste, facilidad de integración y proyección académica.

Tabla de comparación detallada de APIs de LLM

Característica / API	Gemini	OpenAI GPT	DeepSeek	Hugging Face API	Cohere	Claude
Coste y accesibilidad	***	*	**	**	*	*
Calidad y rendimiento	***	***	**	**	**	***
Integración y soporte técnico	***	***	*	**	**	**
Privacidad y cumplimiento (RGPD)	**	**	*	***	**	**
Resultado global	* 11	* 9	* 6	* 9	* 7	* 8

Cuadro 2.3: Comparación de APIs de LLM

2.2– Infraestructura del sistema

La infraestructura técnica del sistema se basa en tecnologías de código abierto y servicios gratuitos que permiten una implementación funcional sin coste. Durante el desarrollo y pruebas del sistema, todos los componentes se ejecutan en entorno local, aunque se contempla la posibilidad de desplegarlos posteriormente en un entorno en la nube.

El framework Rasa se ejecuta en un servidor local utilizando Python como lenguaje base. Las acciones personalizadas del chatbot también están desarrolladas en Python y permiten la integración con otros servicios. Para la comprensión de lenguaje natural, se utiliza la API de Gemini, accedida mediante una clave gratuita proporcionada por Google AI Studio. El almacenamiento y recuperación de datos estructurados y vectoriales se realiza en Supabase, una plataforma que ofrece una base de datos PostgreSQL con soporte para la extensión pgvector. Esto permite centralizar tanto la información relacional (como horarios y datos de profesores) como los vectores semánticos necesarios para las búsquedas por similitud.

La interfaz web del asistente está desarrollada en React y se conecta al backend de Rasa mediante HTTP, funcionando como punto de interacción para el usuario final.

2.2.1. Arquitectura del sistema

La arquitectura del sistema propuesto se basa en un enfoque modular que combina un marco conversacional, técnicas de procesamiento del lenguaje natural mediante modelos generativos, y el uso de bases de datos tanto estructuradas como vectoriales. El sistema está diseñado para ofrecer respuestas útiles a los estudiantes de la Universidad de Sevilla a través de un asistente conversacional accesible desde una interfaz web.

Cuando un usuario realiza una consulta, esta es enviada al motor principal del sistema, construido sobre el framework Rasa, que se encarga de controlar el flujo de la conversación. Para la comprensión de la intención del usuario, el sistema puede operar en dos modos: uno basado en el entrenamiento tradicional de Rasa NLU con ejemplos, y otro utilizando la API del modelo de lenguaje Gemini para detectar la intención de forma dinámica.

Según la intención detectada, el sistema puede acceder a una base de datos estructurada (por ejemplo, para obtener información sobre horarios, profesorado, o fórmulas de calificación) o bien a una base de datos vectorial que contiene fragmentos semánticos extraídos de sitios web oficiales y documentación académica. Esta última permite realizar búsquedas por similitud y obtener información no estructurada de forma precisa.

Si el sistema no logra proporcionar una respuesta satisfactoria, se activa un mecanismo de recuperación (fallback) que consulta una base de datos de enlaces útiles asociados a cada tema. De este modo, se garantiza que el usuario siempre reciba algún tipo de orientación.

Tecnología	Rol en el sistema
Rasa	Framework principal para la gestión del diálogo y flujo de conversación.
Python	Lenguaje principal para el backend y acciones personalizadas.
Gemini (API)	Modelo generativo utilizado para la detección de intención y generación de respuestas naturales.
Supabase	Plataforma de base de datos que permite almacenar información estructurada y vectores semánticos.
pgvector	Extensión de PostgreSQL para el almacenamiento y búsqueda de vectores.
React	Desarrollo de la interfaz web del chatbot.
FAISS (opcional)	Motor local alternativo para búsqueda vectorial (solo si se opta por no usar Supabase).
Google AI Studio	Plataforma utilizada para obtener la API key de Gemini y gestionar el modelo.

Cuadro 2.4: Componentes tecnológicos del sistema

2.2.2. Componentes tecnológicos del sistema

2.3– Casos de uso principales

2.3.1. Consulta sobre horarios de clases

Consultar horario por asignatura

- Escenario positivo: El usuario introduce “*Horario de Bases de Datos II*” y el asistente devuelve su planificación semanal.
- Escenario negativo: El usuario consulta “*Horario de Bases de Datos 7*” (asignatura inexistente) y el asistente informa que no encuentra coincidencias.

Consultar horario por grupo

- Escenario positivo: El usuario pide “*Horario del Grupo A de Ingeniería de Software*” y obtiene la tabla completa.
- Escenario negativo: El usuario introduce un grupo no registrado (“*Grupo Z*”) y el asistente responde que ese grupo no existe.

Consultar horario por carrera/curso

- Escenario positivo: El usuario solicita “*Horario de 3º de Informática*” y recibe la planificación de todas las asignaturas de ese curso.
- Escenario negativo: El usuario introduce “*Horario de 9º de Informática*” y el asistente aclara que no existe tal curso.

2.3.2. Consulta sobre documentación necesaria para trámites

Consultar documentación para matrícula

- Escenario positivo: El usuario pregunta “¿*Qué necesito para matricularme en primero de carrera?*” y el asistente lista DNI, resguardo de pago, etc.
- Escenario negativo: El usuario introduce “*Necesito pasaporte para matrícula de nacional español*” y el asistente aclara que ese requisito no aplica.

Consultar documentación para becas

- Escenario positivo: El usuario pregunta “¿*Qué documentos hacen falta para la beca general?*” y el asistente devuelve la lista oficial (DNI, renta familiar, matrícula).
- Escenario negativo: El usuario consulta “*Puedo solicitar beca solo con una carta de recomendación?*” y el asistente indica que no es suficiente.

Consultar documentación para certificados académicos

- Escenario positivo: El usuario pide “*Documentación necesaria para solicitar certificado de notas?*” y el asistente responde con los requisitos (DNI, impreso de solicitud, pago de tasas).
- Escenario negativo: El usuario pregunta “*Puedo pedir certificado de notas sin identificarme?*” y el asistente informa que no es posible.

Consultar documentación para convalidaciones/traslados

- Escenario positivo: El usuario solicita “*Requisitos para traslado de expediente?*” y el asistente muestra la normativa aplicable.
- Escenario negativo: El usuario introduce “*Convalidar asignaturas sin entregar plan de estudios?*” y el asistente explica que falta documentación esencial.

Comparar trámites similares (opcional)

- Escenario positivo: El usuario consulta “*Diferencia entre certificado simple y compulsado?*” y el asistente explica la distinción.
- Escenario negativo: El usuario pide “*Compulsar sin documento original?*” y el asistente indica que no se puede realizar.

2.3.3. Consulta sobre datos de contacto y tutorías del profesorado**Consultar despacho de un profesor**

- Escenario positivo: El usuario pregunta “*Despacho del profesor Juan Pérez?*” y el asistente devuelve la ubicación.
- Escenario negativo: El usuario introduce un nombre inexistente y el asistente indica que no lo encuentra.

Consultar correo electrónico de un profesor

- Escenario positivo: El usuario pide “*Correo de la profesora Ana López?*” y el asistente devuelve su email institucional.
- Escenario negativo: El usuario solicita “*Correo personal de la profesora Ana López?*” y el asistente aclara que solo puede dar la dirección oficial.

Consultar horario de tutorías

- Escenario positivo: El usuario pregunta “*Horario de tutorías de Miguel García?*” y el asistente devuelve día y hora.
- Escenario negativo: El usuario consulta fuera de rango (“*Tutorías a medianoche?*”) y el asistente indica que no existen en ese horario.

Búsqueda por asignatura

- Escenario positivo: El usuario pregunta “¿Quién da Programación Avanzada?” y el asistente devuelve los profesores con sus datos de contacto.
- Escenario negativo: El usuario pide “Profesor de Programación Interplanetaria” (asignatura inexistente) y el asistente informa del error.

Búsqueda por departamento (opcional)

- Escenario positivo: El usuario solicita “Profesores del Departamento de Lenguajes y Sistemas Informáticos” y obtiene la lista.
- Escenario negativo: El usuario pide un departamento inexistente y el asistente indica que no figura en la base de datos.

2.3.4. Consulta sobre información de una asignatura**Consultar fórmula de calificación**

- Escenario positivo: El usuario pregunta “¿Cómo se califica Ingeniería del Software II?” y el asistente responde “40 % prácticas, 60 % examen”.
- Escenario negativo: El usuario introduce “100 % aprobado automático” y el asistente aclara que no existe esa modalidad.

Consultar sistema de evaluación

- Escenario positivo: El usuario pide “Sistema de evaluación de Matemáticas Discretas” y el asistente devuelve convocatorias y requisitos.
- Escenario negativo: El usuario consulta “Aprobar sin presentarse” y el asistente aclara que no es válido.

Consultar metodologías docentes

- Escenario positivo: El usuario pregunta “Metodología de Programación Avanzada” y el asistente devuelve que es aprendizaje basado en proyectos.
- Escenario negativo: El usuario pide “Metodología libre, sin clases” y el asistente indica que no corresponde con la asignatura.

Consultar bibliografía recomendada

- Escenario positivo: El usuario pide “Bibliografía de Sistemas Operativos” y el asistente devuelve libros y artículos recomendados.
- Escenario negativo: El usuario solicita “Bibliografía de TikTok” y el asistente aclara que no es material académico válido.

Consultar competencias de la asignatura (opcional)

- Escenario positivo: El usuario pregunta “Competencias de Bases de Datos I” y el asistente responde con los objetivos formativos.
- Escenario negativo: El usuario pide “Competencias de Asignatura XYZ inventada” y el asistente aclara que no existe.

2.3.5. Consulta sobre normas y documentación de la Universidad

Consultar convocatorias de examen

- Escenario positivo: El usuario pregunta “*¿Cuántas convocatorias de examen hay en cada asignatura de grado?*” y el asistente responde que, por norma, existen dos por curso.
- Escenario negativo: El usuario pide “*Convocatorias ilimitadas*” y el asistente aclara que no es correcto.

Consultar tiempos de examen

- Escenario positivo: El usuario consulta “*Duración del examen oficial de Matemáticas I*” y el asistente responde que son 2 horas.
- Escenario negativo: El usuario pregunta “*Examen de 10 minutos*” y el asistente indica que ese dato no es válido.

Consultar número máximo de suspensos permitidos

- Escenario positivo: El usuario pide “*¿Cuántas veces se puede suspender una asignatura antes de perder el derecho a examen?*” y el asistente responde que hay un límite (ej. 6).
- Escenario negativo: El usuario consulta “*Suspender infinitas veces*” y el asistente informa que no es posible.

Consultar plazos de revisión de exámenes (opcional)

- Escenario positivo: El usuario pregunta “*¿Cuántos días tengo para pedir revisión de examen tras publicarse las notas?*” y el asistente responde que son 7 días hábiles.
- Escenario negativo: El usuario consulta “*Revisar un examen de hace 3 años*” y el asistente aclara que el plazo ya expiró.

Conceptos iniciales

- 3.1– Sistema conversacional y componentes de Rasa**
- 3.2– Procesamiento del lenguaje natural**
- 3.3– Gestión del diálogo**
- 3.4– Backend y persistencia de datos**

Parte II

Metodología y Planificación

CAPÍTULO 4

Metodología

4.1– SCRUM ajustado en la ejecución

4.2– Herramientas

CAPÍTULO 5

Planificación

5.1– Backlog

5.2– Planificación de sprints

5.3– Cronograma

Referencias

- Autores, V. (2014). *Escuela Técnica Superior de Ingeniería Informática*. ETSII. Descargado de <http://www.informatica.us.es> (fecha de consulta: 24 de Noviembre de 2014)
- Bezoz, J. (2007). The titlesec and titletoc packages. *TexEmplares*, 8, 283–298.
- de Sousa, J. M. (2004). *Ortografía y ortotipografía del español actual*. Trea.